“Reproducing Maia-2: Predicting Human Chess Moves Across Skill Levels”



NAME OF SUPERVISOR:
Sir Qadeer Yaseen

COURSE:
Deep Learning

SUBMITTED BY:
Jawad Mansoor – 232414
Sardar Muhammad Ali Khan– 232967
Muhammad Waqas Siddique - 232441

SUBMISSION DATE: 25/10/2026

Code and results: https://github.com/Jawadyyy/Maia-2-DL-Project

Contents

Abstract 3
1. Introduction 3
2. Paper Selection and Details 3
2.1 Selection criteria 3
2.2 Paper details 4
2.3 Research objective 4
3. Problem Definition and Deep Learning Model 4
3.1 Prediction problem 4
3.2 Model architecture 4
4. Dataset and Preprocessing 5
4.1 Test set (Maia Testset) 5
4.2 Training data 6
4.3 Splits and leakage control 7
5. Implementation and Experimental Setup 7
5.1 Software and hardware environment 7
5.2 Experiments 7
5.3 Model settings and changes from the paper 8
6. Evaluation Results 8
6.1 Reproduction of Table 1 8
6.2 Own training from scratch 9
6.3 Skill-blind analysis (our own) 9
7. Comparison with the Original Paper 9
8. Discussion 11
8.1 Was the experiment reproduced? 11
8.2 Why the numbers differ slightly 11
8.3 Own training: why it is far below the paper 11
8.4 Skill-blind analysis 12
8.5 Limitations 12
8.6 Can the experiment be reproduced from our materials? 12
9. Conclusion 12
References 13
Appendix A. Code Repository and Execution Evidence 14
Execution screenshots 14

Abstract

We reproduce the main move-prediction experiment (Table 1) of *Maia-2: A Unified Model for Human-AI Alignment in Chess* (Tang et al., NeurIPS 2024). Maia-2 combines a ResNet convolutional board encoder with skill-aware attention to predict the move a human player of a given rating will make. Using the authors' official code, released Rapid checkpoint and the paper's 106,740-position test set (with skill-group sizes identical to the paper), we obtain a macro-averaged top-1 accuracy of **53.53%**, against **53.25%** in the paper, a difference of **+0.28 percentage points**. All three per-group accuracies reported in the paper lie inside our 95% confidence intervals, so the experiment is reproduced. We also trained Maia-2 from scratch with the official training code on the 4,041 rated Rapid games available in the official example training file. This reached 17.68% after 3 epochs, more than twice the 7.42% random-guess baseline but far below the paper, which is consistent with having about 41,000× fewer training positions. Finally, a skill-blind analysis (all ratings fixed to 1500) lowers accuracy by 0.76 points, mostly for the weakest and strongest players (McNemar $p < 10^{-14}$ overall), which supports the paper's central claim that conditioning on skill helps.

1. Introduction

Strong chess engines play far better than people, but they are poor models of how people actually play. A human player of a given strength makes characteristic choices and mistakes, and an AI that can predict those choices is useful as a training partner, for analysis and for teaching. Maia-2 addresses this as a supervised deep-learning **prediction (classification) task**: given a board position and the ratings of both players, predict the move the human actually played.

This report reproduces the paper's main result with the authors' released model and data, trains the same architecture ourselves with the official training pipeline, and adds one analysis of our own. It follows the structure of the assignment: paper selection (part b), problem and model (c), data preparation (d), implementation (e), evaluation (f), comparison and visualisation (g), and discussion, limitations and reproducibility (h).

2. Paper Selection and Details

2.1 Selection criteria

Criterion (assignment)	Maia-2	Met
Deep learning research paper	Supervised deep network (ResNet CNN + attention), 23.3M parameters	Yes
Classification / prediction task	Predicts the human move (1,880 possible move classes) for a position and a pair of ratings	Yes
Published in the last 2 years	NeurIPS 2024 proceedings (December 2024); arXiv v2 on 31 Oct 2024	Yes
Paper's dataset available	Lichess open database; the Maia testset and an example training file are released through the official package	Yes
Official code available	github.com/CSSLab/maia2 (MIT licence): training, evaluation and released checkpoints	Yes

2.2 Paper details

Field	Value
Title	Maia-2: A Unified Model for Human-AI Alignment in Chess
Authors	Zhenwei Tang, Difan Jiao, Reid McIlroy-Young, Jon Kleinberg, Siddhartha Sen, Ashton Anderson
Venue	38th Conference on Neural Information Processing Systems (NeurIPS 2024), peer-reviewed
Publication date	December 2024 (NeurIPS 2024). arXiv: v1 30 Sep 2024, v2 31 Oct 2024
DOI / URL	10.48550/arXiv.2409.20553 · https://openreview.net/forum?id=XWlkhRn14K
Official code	https://github.com/CSSLab/maia2, commit 67bee6c (package version 0.11.0)

2.3 Research objective

Earlier work (Maia-1) trained a separate network for each rating band (1100, 1200, …, 1900). These models do not share knowledge, and their predictions are not coherent across skill levels: a stronger model may not play "more strongly" in a consistent way. The objective of Maia-2 is a **single unified model** that takes the ratings of both players as input and, through a **skill-aware attention** mechanism, predicts human moves accurately and coherently across all skill levels. The paper reports that Maia-2 is the most accurate human move predictor across all skill groups (Table 1) while using fewer parameters (23.3M) than the nine Maia-1 models combined (92M).

3. Problem Definition and Deep Learning Model

3.1 Prediction problem

- **Input:** a chess position (FEN), the rating of the player to move and the rating of the opponent. Positions with Black to move are mirrored, so the model always sees the position from the active player's side.
- **Output:** a probability distribution over **1,880 move classes** (every possible from-square/to-square/promotion combination), masked to the legal moves of the position.
- **Target:** the move the human actually played. Top-1 accuracy is the percentage of positions where the most probable legal move equals the played move.
- **Skill groups (paper):** Skilled (< 1600), Advanced (1600–1999) and Master (≥ 2000). The headline "Avg" is the macro-average of the three groups (stated in the paper's Table 1 caption).

3.2 Model architecture

Maia-2 has three parts. The numbers below come from the official configuration (`maia2-training-rapid.yaml`) and the paper's Table 5. Our training log reports **23.30M trainable parameters**, matching the paper's 23.3M.

Component	Details
Board encoding	18 input planes of 8×8 (12 piece planes, side to move, castling rights, en passant)
CNN backbone (ResNet)	Residual convolutional network with 256 channels: an input convolution, 5 residual blocks of two 3×3 convolutions each, and an output convolution (12 convolutional layers in total; the paper lists K_Conv = 12). Output: 8 channels × 8×8.
Patch embedding	Each of the 8 encoded channels (64 values) becomes one token, projected to 1,024 dimensions
Skill-aware attention	2 transformer blocks, 16 heads × 64 dimensions. Learned 128-d embeddings of the active and opponent rating (11 rating categories) are injected into the attention queries.
Heads	Policy head (1,880 move logits), auxiliary head (move/legal-move information), value head (game outcome)
Loss	Cross-entropy (move) + binary cross-entropy (auxiliary) + mean-squared error (value)

4. Dataset and Preprocessing

All data come from the Lichess open database of online games and are downloaded through the official package (`maia2.dataset`), which verifies each file with a SHA-256 checksum.

4.1 Test set (Maia Testset)

Step	Result
Download official example_test_dataset.csv (positions with FEN, played move, both ratings)	127,852 rows from 1,931 games
Paper filter: drop the first 10 plies (move_ply > 10)	106,740 positions (paper Table 6: 106,740)
Group by rating (rounded game rating)	Skilled 56,812 · Advanced 41,747 · Master 8,181, identical to Table 6
Time control check	All 106,740 positions are Rapid games
Positions used in evaluation	106,740 (the full set, no sampling)

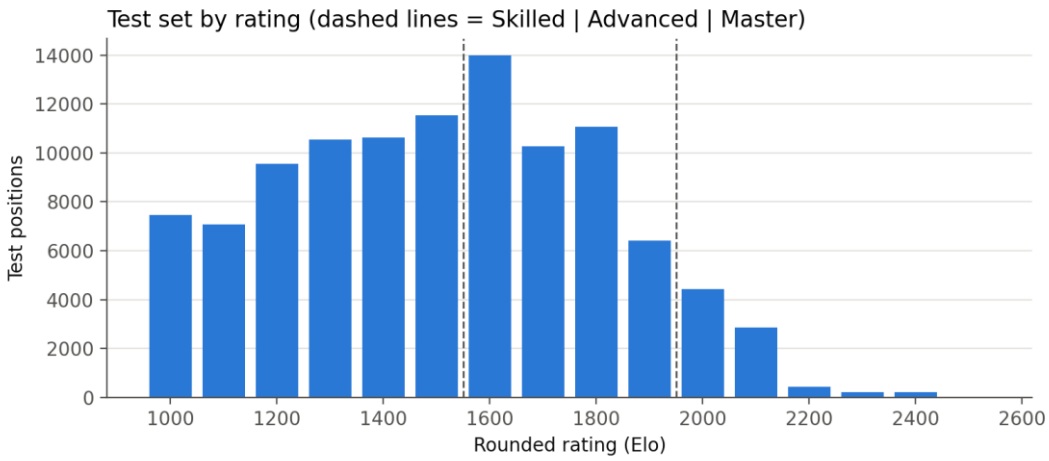


Figure 1. Test positions by rounded rating; dashed lines separate the Skilled, Advanced and Master groups.

4.2 Training data

The paper trained on Lichess Rapid games from January 2013 to November 2023 (excluding December 2019, the test month): 168.93M games and 9.15B positions after filtering (paper Table 7). Downloading and processing this (several terabytes) is impossible on free hardware, so we used the **official example training file** shipped with the code: a Lichess PGN of 28,196 games, all played on 30 April 2018. It contains every time control:

Game type in the file	**Games**
Rated Blitz game	11,227
Rated Bullet game	8,445
Rated Rapid game	3,719
Rated Blitz tournament	2,025
Rated Bullet tournament	1,008
Rated Classical game	865
Rated Rapid tournament	324
Rated UltraBullet game	259
Rated UltraBullet tournament	238
Rated Correspondence game	86
Total	28,196

The official training code (`maia2.main.game_filter` and `process_per_game`) then applies the paper's preprocessing, which we used unchanged:

1. Keep only **rated Rapid** games (Event contains both "Rated" and "Rapid"); drop games played by bot accounts.
2. Keep only games with a valid result, both ratings, and a clock annotation on every move.
3. Skip the first 10 plies of each game and stop after ply 300.
4. Drop positions where the player to move has 30 seconds or less on the clock (time-scramble moves).

5. Mirror positions with Black to move; map each rating to one of 11 categories (< 1100, 1100–1199, …, 1900–1999, ≥ 2000).
6. Encode the board as 18×8×8 planes and the move as one of 1,880 classes; build auxiliary and value targets.

Result: 4,041 games pass the game filter. The trainer reports 3,976 games that contributed at least one position, giving **223,314 training positions** per epoch.

4.3 Splits and leakage control

- **Training split:** the example training file (games from April 2018). **Test split:** the Maia Testset (December 2019 in the paper). The splits are separated in time, exactly as in the paper.
- **Leakage check:** we compared Lichess game IDs between the two files: 0 games are shared.
- **Validation:** the official pipeline has no validation split and trains for a fixed number of epochs (3). We kept this, and did not select checkpoints on the test set. The per-epoch test accuracy is reported for monitoring only, and the final epoch is the reported model.

5. Implementation and Experimental Setup

Everything runs in one Jupyter notebook (`notebooks/maia2_reproduction.ipynb`) on Google Colab. It installs the official package at a pinned commit and calls the authors' functions directly. Our own code only prepares inputs, computes per-group metrics and draws figures.

5.1 Software and hardware environment

Item	Value (from environment.json)
Platform	Google Colab, Linux-6.6.122+-x86_64-with-glibc2.39
GPU	NVIDIA Tesla T4, 15.6 GB · 2 CPU cores
Python / PyTorch / CUDA	3.13.16 / 2.11.0+cu130 / 13.0
Libraries	maia2 0.11.0 (commit 67bee6c), python-chess 1.11.2, NumPy 2.1.3, pandas 2.2.3
Full package list	results/maia2-results-full/pip-freeze.txt

5.2 Experiments

Part	What we ran	Official code used
B: Reproduction (Table 1)	Released Maia-2 Rapid checkpoint on all 106,740 test positions	model.from_pretrained("rapid"), inference.inference_batch
C: Own training	Maia-2 trained from random initialisation, 3 epochs on the example training games	train.run with the official Rapid preset
D: Skill-blind analysis (ours)	The same checkpoint with every rating set to 1500	inference.inference_batch

5.3 Model settings and changes from the paper

Setting	Paper	Ours	Reason
Table 1 model	Maia-2 (Rapid)	Officially released Rapid checkpoint	Same model
Test data	Maia Testset, 106,740 positions	Same file, same filter, same group sizes	None
Training data	168.93M games / 9.15B positions	3,976 games / 223,314 positions	Compute and storage limits
Epochs	Not stated in the paper	3	Value in the official Rapid preset
Batch size	8,192 positions	2,048	16 GB T4 GPU memory
Learning rate / weight decay	1e-4 / 1e-5 (AdamW)	1e-4 / 1e-5 (AdamW)	Same
Balancing cap per rating pair	20 games per chunk of 20,000	Off (all games kept), chunks of 1,000	The cap is designed for monthly files of millions of games; it would discard most of 4k games
Hardware / time	2× A100 80 GB, ~13 days	1× T4, 18.7 min	Free Colab
PyTorch	2.8 (package pin)	2.11.0+cu130	Colab's preinstalled build; package installed without dependencies

The training run used the official `train.run` unchanged. Our wrapper only overrides the configuration values listed above and packs the example PGN in the monthly archive format the trainer expects (`lichess_db_standard_rated_2018-04.pgn.zst`). Each epoch took about 6.1 minutes (3 × 120 optimiser steps); evaluating 106,740 positions took 71–73 seconds on the T4.

6. Evaluation Results

Metrics. The main metric is top-1 move-prediction accuracy, reported per skill group and as the macro-average over the three groups (the paper's "Avg"). We add 95% Wilson confidence intervals, accuracy over all positions (micro-average), top-3 accuracy, and McNemar's paired test for the skill-blind comparison. A uniform random guess among the legal moves would score **7.42%** on this test set (on average 28.8 legal moves per position).

6.1 Reproduction of Table 1

Model	Skilled	Advanced	Master	Avg
Maia 1100 (paper)	51.48	49.13	45.85	48.82
Maia 1500 (paper, best Maia-1)	50.79	52.61	50.76	51.39
Maia 1900 (paper)	48.51	52.26	53.20	51.32

Model	Skilled	Advanced	Master	Avg
Maia-2 (paper)	51.72	54.15	53.87	53.25
Maia-2, reproduced (ours)	**51.98**	**54.44**	**54.16**	**53.53**
95% CI (ours)	51.57–52.39	53.96–54.92	53.08–55.24	—
Difference (ours − paper)	+0.26	+0.29	+0.29	+0.28

6.2 Own training from scratch

Epoch	Positions seen	Skilled	Advanced	Master	Avg	Top-3 avg
1	223,314	11.32	10.02	9.78	10.37	22.39
2	446,628	14.44	13.28	13.96	13.89	29.59
3	669,942	18.23	17.38	17.43	17.68	35.90

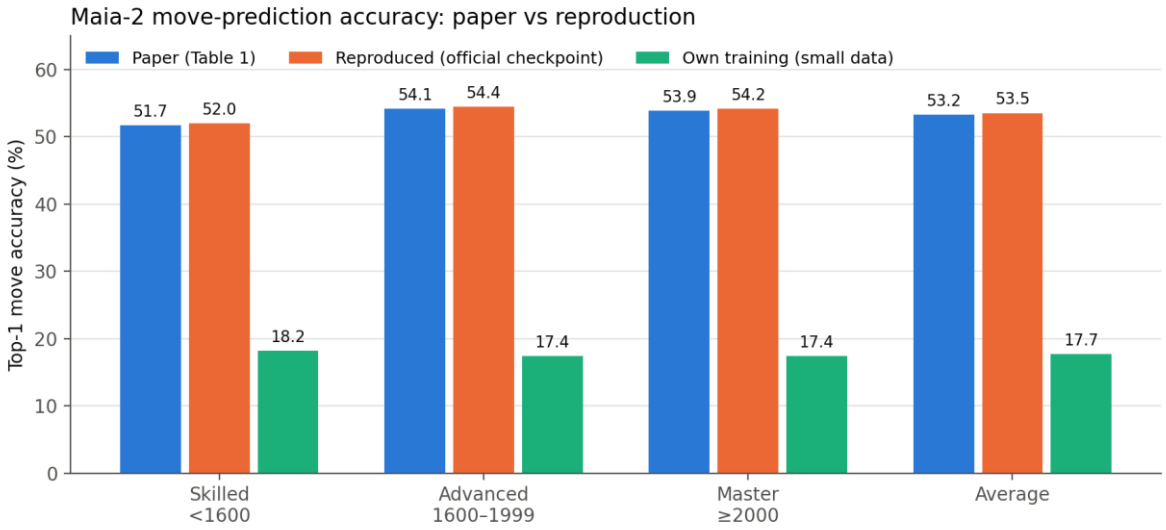
6.3 Skill-blind analysis (our own)

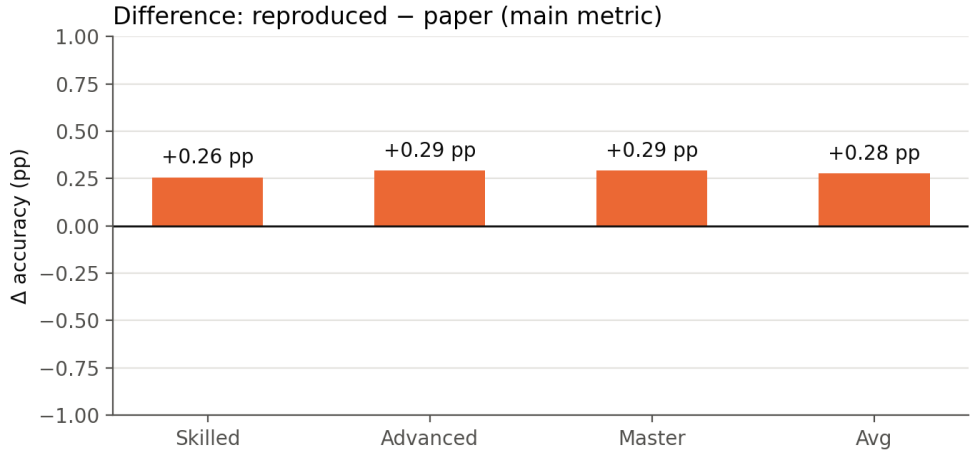
Model	Skilled	Advanced	Master	Avg
Maia-2 checkpoint, real ratings	51.98	54.44	54.16	53.53
Same checkpoint, all ratings = 1500	51.43	54.15	52.72	52.77
Drop without rating information	−0.54	−0.29	−1.44	−0.76
McNemar p-value (paired)	1.7×10^{-10}	4.6×10^{-3}	2.7×10^{-5}	5.7×10^{-15} (all)

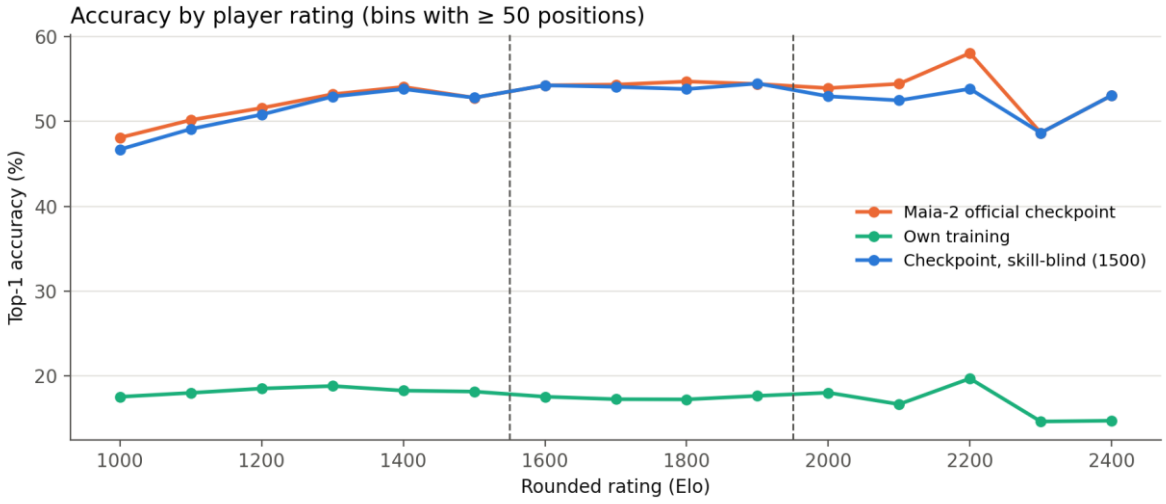
By rating band, removing the ratings costs 1.23 points for players below 1200, 0.30 points for 1200–1999, and 1.44 points for 2000 and above.

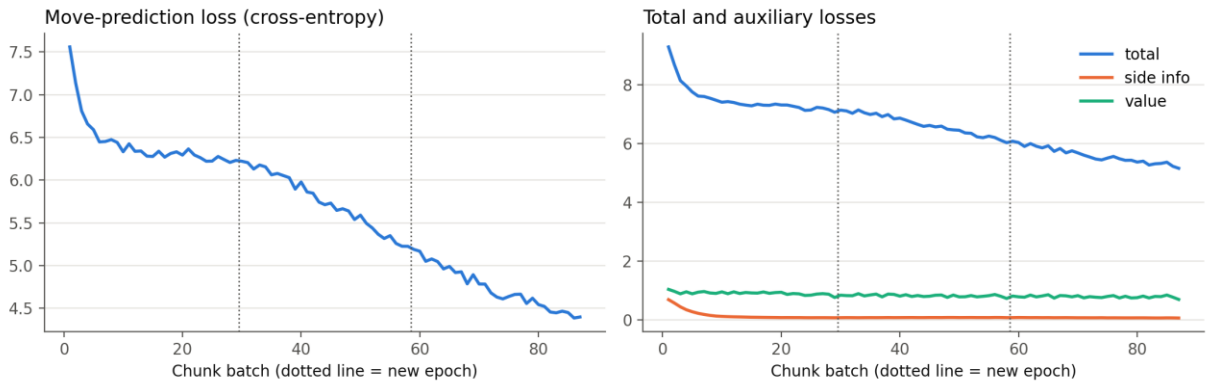
7. Comparison with the Original Paper

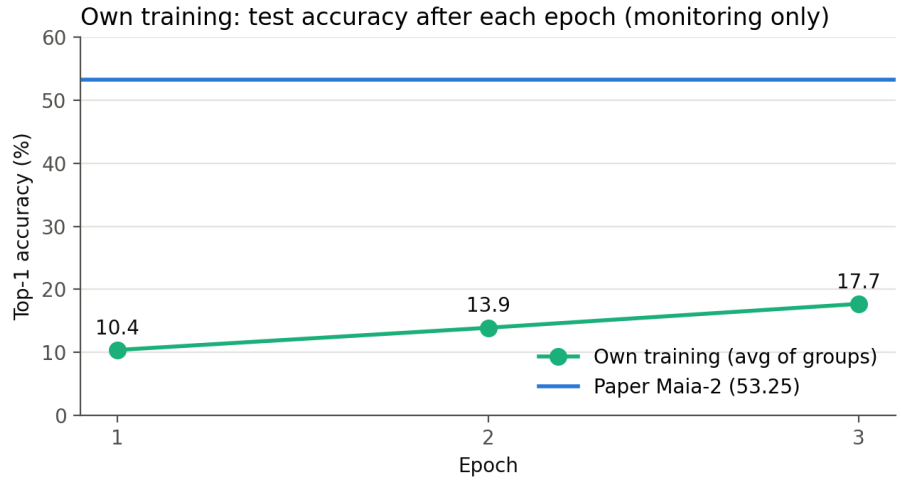
Main metric difference. Difference = reproduced − paper. For the macro-averaged top-1 accuracy: 53.53 − 53.25 = **+0.28 percentage points**. The per-group differences are Skilled +0.26, Advanced +0.29, Master +0.29 pp.


Maia-2 move-prediction accuracy: paper vs reproduction
Paper (Table 1)
Reproduced (official checkpoint)
Own training (small data)
Top-1 move accuracy (%)
0
10
20
30
40
50
60
51.7
52.0
18.2
54.1
54.4
17.4
53.9
54.2
17.4
53.2
53.5
17.7
Skilled
<1600
Advanced
1600–1999
Master
≥2000
Average


Difference: reproduced − paper (main metric)
Δ accuracy (pp)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
+0.26 pp
+0.29 pp
+0.29 pp
+0.28 pp
Skilled
Advanced
Master
Avg


Accuracy by player rating (bins with ≥ 50 positions)
Top-1 accuracy (%)
20
30
40
50
60
1000
1200
1400
1600
1800
2000
2200
2400
Rounded rating (Elo)
Maia-2 official checkpoint
Own training
Checkpoint, skill-blind (1500)





8. Discussion

8.1 Was the experiment reproduced?

Yes. The reproduced accuracies differ from the paper by at most 0.29 points, and every paper value lies inside our 95% confidence interval (Skilled 51.57–52.39, Advanced 53.96–54.92, Master 53.08–55.24). The ranking in Table 1 is also preserved: our reproduced Maia-2 beats every Maia-1 model in every group, by 2.14 points on average over the best one (Maia 1500).

8.2 Why the numbers differ slightly

- **Sampling noise.** With 8,181 Master positions the 95% interval is about ±1.1 points; a 0.3-point difference is well within it.
- **Released checkpoint versus paper-time model.** We evaluated the publicly released weights. The paper does not state that its Table 1 numbers were computed with exactly this file, and a later re-export would shift results by small amounts.
- **Software and hardware.** We used PyTorch 2.11 on a T4 instead of the authors' environment. Floating-point differences in GPU kernels can change near-tied predictions.
- **Tie-breaking.** The official inference ranks moves by unrounded probability; any differences in how the original evaluation broke ties would also move results by a few hundredths of a point.

8.3 Own training: why it is far below the paper

Our from-scratch model reached 17.68% (macro-average) after 3 epochs, about 2.4× the 7.42% random baseline but 35.57 points below the paper. The causes are clear from the setup:

- **Data scale.** 223,314 unique training positions versus 9.15 billion in the paper, about 41,000× fewer, all from a single day of play.
- **Training length.** The move loss fell from 7.56 to 4.40 and was still falling (Figure 5), and accuracy rose every epoch (10.37 → 13.89 → 17.68%). The model had not converged.
- **Changed settings.** A smaller batch (2,048) and no data balancing; with so little data, balancing would have removed most games.

This experiment shows that the official training pipeline works and learns, but reproducing the paper's training result requires the paper's data and compute (2× A100 for about 13 days), which was outside our means.

8.4 Skill-blind analysis

Hiding the ratings lowers accuracy by 0.76 points on average. The effect is statistically significant (McNemar $p = 5.7\times10^{-15}$) and is concentrated where a player differs most from the 1500 default: 1.23 points below 1200 and 1.44 points at 2000 and above, against 0.30 points in between (Figure 4). This supports the paper's claim that conditioning on skill makes predictions more human-like for each level. It is our own test, not the paper's Table 3 ablation, which retrains the model without the skill-aware attention.

8.5 Limitations

- Only the paper's main experiment (Table 1) was reproduced. The cross-skill, perplexity (Grounded Testset, Table 2) and Table 3 ablation experiments need test sets or training runs we could not obtain or afford.
- The reproduction evaluates the released checkpoint; we could not retrain the full model to confirm the training procedure itself at scale.
- Our own training used one seed, one day of games from 2018, and only 3 epochs.
- Lichess players are online players; results may not transfer to over-the-board or titled play.
- The paper is inconsistent about the test-set name: the Table 1 caption says "Maia-2 Testset", while Section 4.1 and Table 6 call the same 106,740-position set the "Maia-1 Testset". Our group sizes match Table 6 exactly.
- Software versions differ from the package pin (PyTorch 2.11.0+cu130 instead of 2.8, Python 3.13.16 instead of ≤ 3.12).

8.6 Can the experiment be reproduced from our materials?

Yes. The repository (https://github.com/Jawadyyy/Maia-2-DL-Project) contains the notebook, which pins the official code to a fixed commit and downloads the checksum-verified data and checkpoint automatically. A reader needs only a free Google Colab account with a T4 GPU and about one hour. No credentials are needed. The repository also contains every output of our run: the environment record, the timestamped training log, all predictions, the result tables and the figures. Every number in this report was read from those files.

9. Conclusion

We reproduced the main result of Maia-2 (NeurIPS 2024): with the authors' code, checkpoint and test set, the macro-averaged human move-prediction accuracy is 53.53% against the reported 53.25% (+0.28 pp), and every per-group value lies within our confidence intervals. Training the same CNN-plus-attention architecture ourselves with the official pipeline worked but, with about 0.002%

of the paper's data, reached 17.68%, showing that the paper's result depends heavily on data scale. Our skill-blind test confirms that the rating input contributes measurably, especially for the weakest and strongest players.

References

[1] Z. Tang, D. Jiao, R. McIlroy-Young, J. Kleinberg, S. Sen and A. Anderson, "Maia-2: A Unified Model for Human-AI Alignment in Chess," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. https://openreview.net/forum?id=XWlkhRn14K, doi:10.48550/arXiv.2409.20553.

[2] CSSLab, University of Toronto, "Maia-2: official implementation," GitHub, MIT licence, commit 67bee6c. https://github.com/CSSLab/maia2

[3] Lichess, "lichess.org open database." https://database.lichess.org/

[4] R. McIlroy-Young, S. Sen, J. Kleinberg and A. Anderson, "Aligning Superhuman AI with Human Behavior: Chess as a Model System," in *Proc. 26th ACM SIGKDD*, 2020. doi:10.1145/3394486.3403219.

[5] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE CVPR*, 2016. doi:10.1109/CVPR.2016.90.

[6] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, 12(2), 153–157, 1947.

Appendix A. Code Repository and Execution Evidence

Repository: https://github.com/Jawadyyy/Maia-2-DL-Project

Path	Contents
notebooks/maia2_reproduction.ipynb	The full experiment, saved with its Colab outputs from the reported run
notebooks/maia2_reproduction_smoke_run.ipynb	Pipeline check run (small sample), not used for results
results/maia2-results-full/	environment.json, pip-freeze.txt, data_summary.json, training_overrides.json, train.log, predictions_*.csv, result tables, figures/
paper/	The original paper (PDF)
references.bib	Citations for the paper, code and data

Excerpts from the run logs (copied unchanged from the notebook output and train.log):

```
[notebook] Raw test rows: 127852 | games: 1931 | after move_ply > 10: 106740
[notebook] Group sizes: {'Skilled': 56812, 'Advanced': 41747, 'Master': 8181} | paper:
{'Skilled': 56812, 'Advanced': 41747, 'Master': 8181}
[notebook] Train PGN: 28196 games, 4041 pass the official filter, ~223,314 training positions;
dates ['2018.04.30']; audit 81s
[notebook] Games shared between train and test: 0
[notebook] official inference_batch accuracy 0.5311 | 106740 positions in 73s
[notebook] exit code 0 | training wall time 18.7 min
[notebook] Epoch 3: trained on 669,942 positions / 11,928 games
[train.log] 07:17:13  device: cuda
[train.log] 07:17:18 Trainable Parameters: 23.30M
[train.log] 07:23:30 Month totals: positions=223314, games=3976, optimizer_steps=120
[train.log] 07:23:30 ### (1 / 1) Took 6m 11.36s to train
/content/work/lichess_data/lichess_db_standard_rated_2018-04.pgn with 29 chunks.
[train.log] 07:29:41 Month totals: positions=223314, games=3976, optimizer_steps=120
[train.log] 07:29:41 ### (1 / 1) Took 6m 9.19s to train
/content/work/lichess_data/lichess_db_standard_rated_2018-04.pgn with 29 chunks.
[train.log] 07:35:48 Month totals: positions=223314, games=3976, optimizer_steps=120
[train.log] 07:35:48 ### (1 / 1) Took 6m 6.02s to train
/content/work/lichess_data/lichess_db_standard_rated_2018-04.pgn with 29 chunks.
```

Execution screenshots

Screenshots 1–6 are rendered from the outputs saved in the executed notebook (`notebooks/maia2_reproduction.ipynb`), which ran on Google Colab with a Tesla T4 GPU on 25 October 2026. No output was edited; where a long output is shortened, the omitted part is marked in the image and is available in full in the repository. Screenshot 7 shows the repository's git history and files. The same images are in `docs/screenshots/`.

maia2_reproduction.ipynb · Software environment and GPU — Google Colab run (Tesla T4), 25 Oct 2026, rendered from the saved notebook outputs

```
[3]     import importlib.metadata as md_
        import torch, numpy as np, pandas as pd, chess
        env = dict(
            python=sys.version.split()[0], platform=platform.platform(),
            torch=torch.__version__, cuda=torch.version.cuda,
            gpu=torch.cuda.get_device_name(0) if torch.cuda.is_available() else "none",
            gpu_memory_gb=round(torch.cuda.get_device_properties(0).total_memory / 1e9, 1) if torch.cuda.is_available() else None,
            cpu_count=os.cpu_count(), numpy=np.__version__, pandas=pd.__version__,
            python_chess=md_.version("chess"), maia2=md_.version("maia2"), maia2_commit=MAIA2_COMMIT,
            colab=IN_COLAB, smoke_test=SMOKE_TEST,
        # … 7 more lines (full source in the notebook)
```


```
{
  "python": "3.13.16",
  "platform": "Linux-6.6.122+-x86_64-with-glibc2.39",
  "torch": "2.11.0+cu130",
  "cuda": "13.0",
  "gpu": "Tesla T4",
  "gpu_memory_gb": 15.6,
  "cpu_count": 2,
  "numpy": "2.1.3",
  "pandas": "2.2.3",
  "python_chess": "1.11.2",
  "maia2": "0.11.0",
  "maia2_commit": "67bee6ce7e2f264344fe6d2026a2e18c76a82efa",
  "colab": true,
  "smoke_test": false,
  "settings": {
    "epochs": 3,
    "batch_size": 2048,
    "chunk_size": 1000,
    "balance_cap": null,
    "seed": 42
  }
}
```


```
Sun Oct 25 07:14:11 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   45C    P8             12W /   70W |       3MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Screenshot 1. Software environment record and nvidia-smi output (Tesla T4).

maia2_reproduction.ipynb · Dataset download, filters, splits and leakage check Google Colab run (Tesla T4), 25 Oct 2026, rendered from the saved notebook outputs

```
[4]  from maia2 import dataset
     from maia2.main import game_filter
     from maia2.utils import extract_clock_time
     import chess.pgn

     DATA = WORK / "maia2_data"
     official_test = dataset.load_example_test_dataset(save_root=str(DATA))   # official loader: download + checksum + ply filter
     raw_test = pd.read_csv(DATA / "example_test_dataset.csv")
     test = raw_test[raw_test.move_ply > 10].reset_index(drop=True)            # identical filter, keeping the metadata columns
     assert len(test) == len(official_test)
     # … 14 more lines (full source in the notebook)
```

```
Downloading...
From: https://drive.google.com/uc?id=1fSu4Yp8uYj7xocbHAbjBP6DthsgiJy9X
To: /content/work/maia2_data/.example_test_dataset.csv.w1dcpvma.part
100%|██████████| 22.5M/22.5M [00:00<00:00, 56.3MB/s]
```

```
Raw test rows: 127852 | games: 1931 | after move_ply > 10: 106740
Group sizes: {'Skilled': 56812, 'Advanced': 41747, 'Master': 8181} | paper: {'Skilled': 56812, 'Advanced': 41747, 'Master': 8181}
Time controls: {'Rapid': 106740}
```

```
[5]  train_pgn = Path(dataset.load_example_train_dataset(save_root=str(DATA)))

     def train_data_audit(path, limit=None):
         """Apply the official game filter and position filters to count what the training code will use."""
         stats = dict(games=0, kept_games=0, positions_after_filters=0, dates=set(), game_ids=set())
         with open(path, encoding="utf-8") as fh:
             while (game := chess.pgn.read_game(fh)) is not None:
                 stats["games"] += 1
     # … 29 more lines (full source in the notebook)
```

```
Downloading...
From: https://drive.google.com/uc?id=1XBeuhB17z50mFK4tDvPG9rQRbxLSzNqB
To: /content/work/maia2_data/.example_train_dataset.csv.t8le4813.part
100%|██████████| 66.2M/66.2M [00:01<00:00, 37.5MB/s]
```

```
Train PGN: 28196 games, 4041 pass the official filter, ~223,314 training positions; dates ['2018.04.30']; audit 81s
Games shared between train and test: 0

527
```

Screenshot 2. Dataset download and checks: test-group sizes identical to the paper, training-data audit, zero train/test overlap.

maia2_reproduction.ipynb · Table 1 reproduction Google Colab run (Tesla T4), 25 Oct 2026, rendered from the saved notebook outputs

```
[7]  from maia2 import model as maia_model, inference
     from statsmodels.stats.proportion import proportion_confint

     PAPER_T1 = {  # Tang et al. (NeurIPS 2024), Table 1, top-1 accuracy (%) on the Maia Testset
         "Maia-2": dict(Skilled=51.72, Advanced=54.15, Master=53.87, Avg=53.25),
         "Maia 1500 (best Maia-1)": dict(Skilled=50.79, Advanced=52.61, Master=50.76, Avg=51.39),
     }

     # … 42 more lines (full source in the notebook)
```

```
Downloading or validating the model for rapid games.
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1gbC1-c7c0EQOPPAVpGWubezeEW8grVwc
From (redirected): https://drive.google.com/uc?id=1gbC1-c7c0EQOPPAVpGWubezeEW8grVwc&confirm=t&uuid=e195443b-7683-4efd-b959-db3782c6a744
To: /content/work/models/.rapid_model.pt.vtysde9z.part
100%|██████████| 280M/280M [00:09<00:00, 28.2MB/s]
```

```
Model for rapid games loaded to cuda.
```

```
100%|██████████| 105/105 [01:08<00:00,  1.54it/s]
```

```
official inference_batch accuracy 0.5311 | 106740 positions in 73s
```

	Model	Skilled	Advanced	Master	Avg	Δ Avg vs paper (pp)	Overall (all positions)	Top-3 avg
0	Maia-2 (paper)	51.72	54.15	53.87	53.25	0.00	NaN	NaN
1	Maia 1500 (best Maia-1) (paper)	50.79	52.61	50.76	51.39	-1.86	NaN	NaN
2	Maia-2 official checkpoint (reproduced)	51.98	54.44	54.16	53.53	0.28	53.11	81.52

	Skilled	Advanced	Master
95% CI (reproduced)	51.57–52.39	53.96–54.92	53.08–55.24

Screenshot 3. Table 1 reproduction with the official checkpoint on all 106,740 test positions.

maia2_reproduction.ipynb · Own training with the official trainer — Google Colab run (Tesla T4), 25 Oct 2026, rendered from the saved notebook outputs

```
[9]  log_path = OUT / "train.log"
     t0 = time.time()
     with open(log_path, "w") as log:
         proc = subprocess.Popen([sys.executable, "train_maia2.py", json.dumps(overrides)],
                                 stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
         for line in proc.stdout:
             line = f"{time.strftime('%H:%M:%S')} {line}"
             log.write(line); log.flush()
     # … 6 more lines (full source in the notebook)
```

```
07:17:13 Configurations:
07:17:13 Training Data:
07:17:13 /content/work/lichess_data/lichess_db_standard_rated_2018-04.pgn
07:17:13 MAIA2Model(
07:17:18 Trainable Parameters: 23.30M
07:17:18 Epoch 1
… 435 lines omitted here (full log in results/maia2-results-full/train.log) …
07:35:20 [26/29]
07:35:20 [# Positions]: 653.72K
07:35:20 [# Games]: 11.62K
07:35:20 [# Loss]: 5.321 | [# Loss MAIA]: 4.467 | [# Loss Side Info]: 0.064 | [# Loss Value]: 0.79
07:35:23 {(0, 0): 9, (7, 7): 8, (4, 3): 10, (2, 2): 8, (7, 6): 7, (6, 4): 4, (5, 4): 5, (8, 7): 4, (3, 2): 7, (8, 8): 8, (1, 1): 3, (9, 3): 1, (2, 0):
1, (5, 0): 1, (4, 4): 8, (7, 5): 5, (3, 3): 3, (2, 1): 2, (1, 0): 2, (8, 5): 1, (6, 6): 6, (6, 5): 4, (10, 10): 3, (6, 3): 1, (5, 3): 2, (10, 9): 2,
(5, 5): 7, (9, 9): 2, (3, 1): 2, (9, 8): 1, (7, 4): 1, (5, 2): 1, (8, 2): 1}
07:35:32 [27/29]
07:35:32 [# Positions]: 660.64K
07:35:32 [# Games]: 11.75K
07:35:32 [# Loss]: 5.363 | [# Loss MAIA]: 4.451 | [# Loss Side Info]: 0.067 | [# Loss Value]: 0.844
07:35:35 {(6, 5): 13, (6, 6): 13, (8, 7): 3, (5, 5): 12, (7, 5): 3, (3, 2): 7, (3, 3): 10, (2, 1): 3, (4, 3): 4, (5, 4): 7, (8, 4): 2, (9, 9): 2, (7,
7): 7, (8, 8): 3, (8, 5): 1, (10, 10): 2, (0, 0): 6, (2, 2): 6, (2, 0): 1, (10, 6): 1, (5, 3): 2, (6, 3): 1, (1, 0): 5, (4, 4): 8, (7, 4): 3, (9, 8):
4, (4, 1): 1, (5, 1): 1, (7, 6): 4, (9, 7): 3, (9, 3): 1, (8, 6): 2, (1, 1): 3, (10, 9): 1, (6, 0): 1, (6, 4): 1, (4, 2): 1, (5, 2): 2}
07:35:45 [28/29]
07:35:45 [# Positions]: 668.17K
07:35:45 [# Games]: 11.90K
07:35:45 [# Loss]: 5.225 | [# Loss MAIA]: 4.387 | [# Loss Side Info]: 0.066 | [# Loss Value]: 0.772
07:35:45 {(8, 4): 1, (1, 0): 2, (6, 5): 2, (9, 7): 1, (5, 5): 2, (4, 1): 1, (1, 1): 2, (5, 4): 1, (0, 0): 2, (2, 2): 1, (4, 3): 2, (4, 4): 2, (7, 6):
2, (10, 8): 1, (3, 2): 3, (7, 7): 1, (8, 8): 2, (2, 1): 1, (10, 7): 1, (8, 6): 1}
07:35:48 [29/29]
07:35:48 [# Positions]: 669.94K
07:35:48 [# Games]: 11.93K
07:35:48 [# Loss]: 5.155 | [# Loss MAIA]: 4.399 | [# Loss Side Info]: 0.062 | [# Loss Value]: 0.694
07:35:48 Month totals: positions=223314, games=3976, optimizer_steps=120
07:35:48 ### (1 / 1) Took 6m 6.02s to train /content/work/lichess_data/lichess_db_standard_rated_2018-04.pgn with 29 chunks.
exit code 0 | training wall time 18.7 min
```

Screenshot 4. Own training with the official trainer: start of the log, final chunks of epoch 3 and exit code 0.

maia2_reproduction.ipynb · Own-trained model evaluated after each epoch — Google Colab run (Tesla T4), 25 Oct 2026, rendered from the saved notebook outputs

```
[11] from maia2 import utils as mutils
     from maia2.main import MAIA2Model
     from maia2.train import load_model_state_dict
     from importlib.resources import as_file, files

     with as_file(files("maia2.configs").joinpath("maia2-training-rapid.yaml")) as path:
         train_cfg = mutils.parse_args(path)
     for k, v in overrides.items():
     # … 32 more lines (full source in the notebook)
```

```
Epoch 1: trained on 223,314 positions / 3,976 games
100%|██████████| 105/105 [01:07<00:00,  1.55it/s]
official inference_batch accuracy 0.1069 | 106740 positions in 72s
Epoch 2: trained on 446,628 positions / 7,952 games
100%|██████████| 105/105 [01:07<00:00,  1.56it/s]
official inference_batch accuracy 0.1395 | 106740 positions in 72s
Epoch 3: trained on 669,942 positions / 11,928 games
100%|██████████| 105/105 [01:07<00:00,  1.57it/s]
official inference_batch accuracy 0.1784 | 106740 positions in 71s
```

	epoch	positions_seen	Skilled	Advanced	Master	Avg	Top-3 avg
0	1	223314	11.32	10.02	9.78	10.37	22.39
1	2	446628	14.44	13.28	13.96	13.89	29.59
2	3	669942	18.23	17.38	17.43	17.68	35.90

Screenshot 5. The own-trained model evaluated on the full test set after each epoch.

maia2_reproduction.ipynb · Final comparison with the paper — Google Colab run (Tesla T4), 25 Oct 2026, rendered from the saved notebook outputs

```
[13]  paper = PAPER_T1["Maia-2"]
      final = pd.DataFrame([{"Model": "Maia-2 (paper, Table 1)", **paper}, pre_row, own_row, blind_row])
      for g in GROUPS + ["Avg"]:
          final[f"Δ {g} (pp)"] = final[g] - paper[g]
      final.to_csv(OUT / "final_comparison.csv", index=False)
      display(final[["Model", *GROUPS, "Avg", *[f"Δ {g} (pp)" for g in GROUPS + ["Avg"]]]].round(2))
      summary = dict(main_metric="top-1 accuracy, mean of skill groups", paper_avg=paper["Avg"],
                     reproduced_avg=round(pre_row["Avg"], 2), difference_pp=round(pre_row["Avg"] - paper["Avg"], 2),
      # … 4 more lines (full source in the notebook)
```

	Model	Skilled	Advanced	Master	Avg	Δ Skilled (pp)	Δ Advanced (pp)	Δ Master (pp)	Δ Avg (pp)
0	Maia-2 (paper, Table 1)	51.72	54.15	53.87	53.25	0.00	0.00	0.00	0.00
1	Maia-2 official checkpoint (reproduced)	51.98	54.44	54.16	53.53	0.26	0.29	0.29	0.28
2	Own training from scratch (final epoch 3)	18.23	17.38	17.43	17.68	-33.49	-36.77	-36.44	-35.57
3	Maia-2 checkpoint, ratings fixed to 1500 (skil...	51.43	54.15	52.72	52.77	-0.29	-0.00	-1.15	-0.48


```
{
  "main_metric": "top-1 accuracy, mean of skill groups",
  "paper_avg": 53.25,
  "reproduced_avg": 53.53,
  "difference_pp": 0.28,
  "own_training_avg": 17.68,
  "skill_blind_avg": 52.77,
  "training_minutes": 18.7,
  "smoke_test": false
}
```


Screenshot 6. Final comparison table with differences from the paper.

Maia-2-DL-Project · history and files — git output of the team repository (private GitHub repo)

```
$       git remote get-url origin
        git log
        git ls-tree -r --name-only HEAD
```

```
https://github.com/Jawadyyy/Maia-2-DL-Project.git

98b0ffc  2026-10-25 18:11  Jawadyyyy  Add reproduction report (.docx)
0071f04  2026-10-25 12:54  Muhammad Waqas Siddique  ran notebook and added results in output
6f0c338  2026-10-25 03:26  Jawadyyyy  Add Colab smoke-test run outputs
d231211  2026-10-24 22:50  Jawadyyyy  Organise repo: paper, notebooks, results, docs folders
b520769  2026-10-24 22:29  Jawadyyyy  Add Maia-2 reproduction notebook and README

.gitignore
README.md
docs/report/Maia2_Reproduction_Report.docx
docs/screenshots/.gitkeep
notebooks/maia2_reproduction.ipynb
notebooks/maia2_reproduction_smoke_run.ipynb
paper/Maia-2_Tang_et_al_NeurIPS2024.pdf
references.bib
results/README.md
results/figures/.gitkeep
results/maia2-results-full/data_summary.json
results/maia2-results-full/environment.json
results/maia2-results-full/figures/accuracy_by_rating.png
results/maia2-results-full/figures/difference_vs_paper.png
results/maia2-results-full/figures/own_training_accuracy_by_epoch.png
results/maia2-results-full/figures/paper_vs_reproduced.png
results/maia2-results-full/figures/test_rating_distribution.png
results/maia2-results-full/figures/training_loss.png
results/maia2-results-full/final_comparison.csv
results/maia2-results-full/own_training_by_epoch.csv
results/maia2-results-full/pip-freeze.txt
results/maia2-results-full/summary.json
results/maia2-results-full/table1_reproduction.csv
results/maia2-results-full/train.log
results/maia2-results-full/training_curve.csv
results/maia2-results-full/training_overrides.json
(also: results/smoke_test/* and results/maia2-results-full/predictions_*.csv)
```

Screenshot 7. Team repository: remote URL, commit history and tracked files.